Wrocław
University
of Science
and Technology

Software Engineering

Requirements Engineering: traditional approach.

Marcin Jodłowiec

October 22nd, 2025

Wrocław University of Science and Technology

Agenda

- Definitions
  - What is Requrement?
  - What is Requirements Engineering?
  - Core activities
- Requirement types
  - Non-functional requirements taxonomy
  - FURPS model
- Requirements Elicitation
  - Understanding the context
  - Requirement sources
  - The Kano model
  - Requirement elicitation techniques
- Requirements specification
  - Requirement specification techniques
  - Glossary design
  - Natural language implications
- Model-based requirements specification
  - Use case diagrams
- Requirements validation
  - Validation principles
  - Validation techniques
- Requirements management
  - Requirements prioritization techniques
  - Traceability and change management

Wrocław
University
of Science
and Technology

Definitions

What is Requrement?

Requirement

1. A condition or capability needed by a user to **solve a problem** or **achieve an objective**.
2. A condition or capability that must be met or possessed by a system or system component to **satisfy a contract**, **standard**, **specification**, or other **formally imposed document**.
3. A documented representation of a condition or capability as in (1) or (2).

(definition from IEEE Std 610.12-1990)

What is Requirements Engineering?

Requirements Engineering

1. Requirements engineering is a **systematic** and disciplined approach to the **specification** and **management** of requirements with the following goals:
   - 1.1 Knowing the relevant requirements, achieving a consensus among the stakeholders about these requirements, documenting them according to given standards, and managing them systematically
   - 1.2 **Understanding and documenting** the stakeholders' desires and needs, they specifying and managing requirements to minimize the risk of delivering a system that does not meet the stakeholders' desires and needs

(definition from IEEE Std 610.12-1990)

Core activities of RE

- **Elicitation** – also called requirements discovery, requirements gathering, or requirements acquisition
- **Elaboration** – the process of analyzing and refining the requirements
- **Negotiation** – the process of reaching a consensus among the stakeholders on the requirements
- **Specification** – the process of documenting the requirements
- **Validation** and verification – checking that the requirements define the system that the customer really wants
- **Management** – the process of managing changing requirements during the requirements engineering process and system development

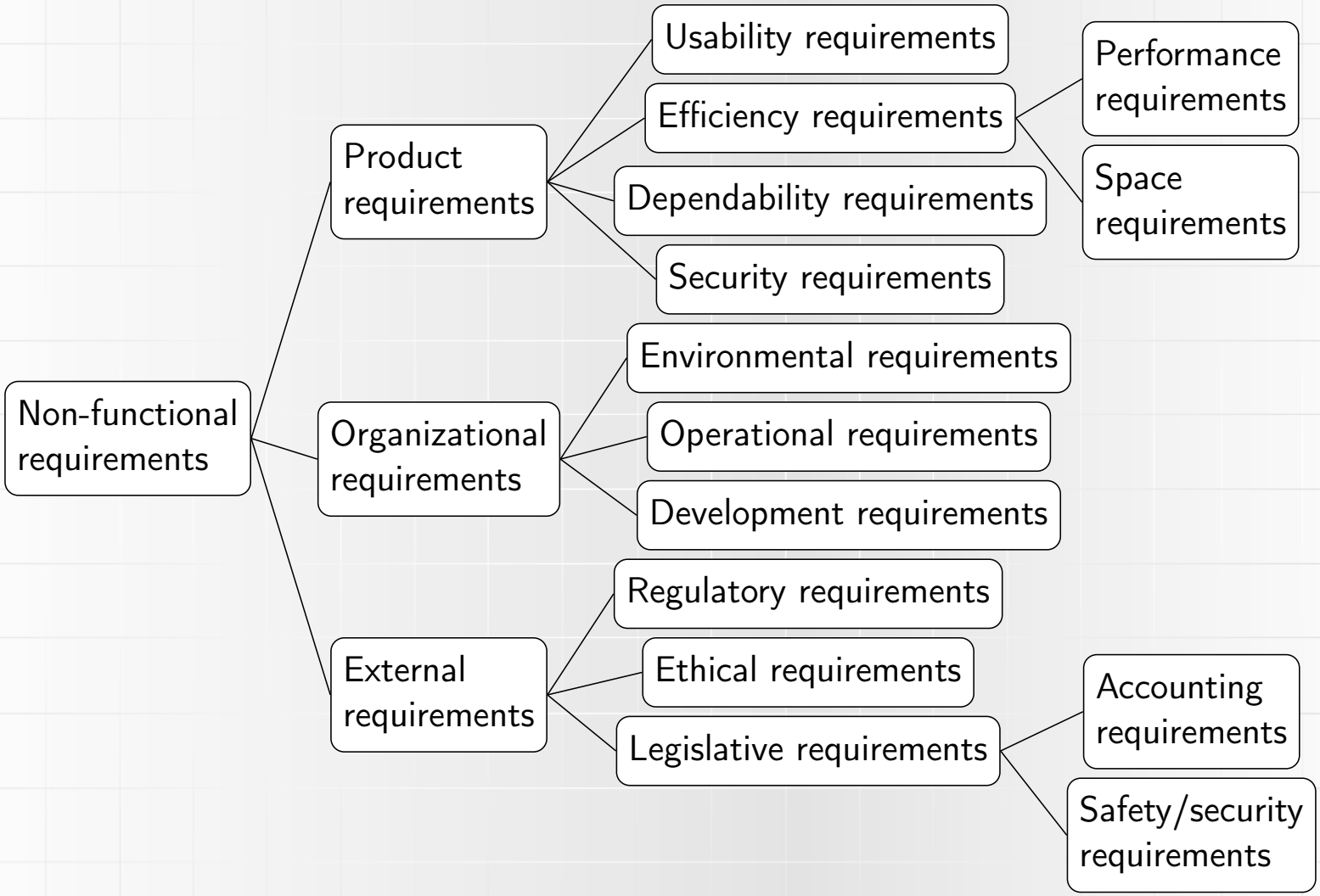
Wrocław
University
of Science
and Technology

Requirement types

Requirement types

- **Functional requirements** – concern a result of **behavior** that shall be provided by the **function** of a system.
  - Example: "The system shall send an email notification when a new user registers."
- **Non-functional requirements**: quality requirements or constraints
- **Quality requirements** – specify quality attributes, i.a. **performance**, **usability**, **reliability**, **security**, portability, availability, . . .
  - Example: "The system shall respond to user queries within 2 seconds."
  - Example: "The system shall be available 99.9% of the time."
- **Constraints** – restrictions on the design or implementation of the system, e.g., development process, regulatory policies, hardware limitations, etc.
  They are not implemented – they limit the solution space available during the development process.
  - Example: "The system shall be developed using the Java programming language."
  - Example: "The system shall comply with GDPR regulations."

Non-functional requirements taxonomy



FURPS model

Definition

FURPS is a classification model for **non-functional requirements** proposed by Hewlett-Packard. It organizes quality attributes into five major categories.

F – Functionality	Security, Interoperability, Accuracy, Feature set
U – Usability	Human factors, Aesthetics, Consistency, Learnabili
R – Reliability	Availability, Failure rate, Recovery, MTBF[1]
P – Performance	Response time, Throughput, Resource usage, Scal
S – Supportability	Maintainability, Extensibility, Portability, Testabilit

(Grady & Caswell, Hewlett-Packard, 1992)

[1] Mean TIme Between Failures

Wrocław
University
of Science
and Technology

Requirements Elicitation

Understanding the context I

System context

The system context is the part of the system environment that is relevant for the definition as well as the understanding of the requirements of a system to be developed

- People, other systems, processes, events, documents...
- completness is crucial $\rightarrow$ system boundary
- System boundary: separation between the system to be developed and its environment

Understanding the context II

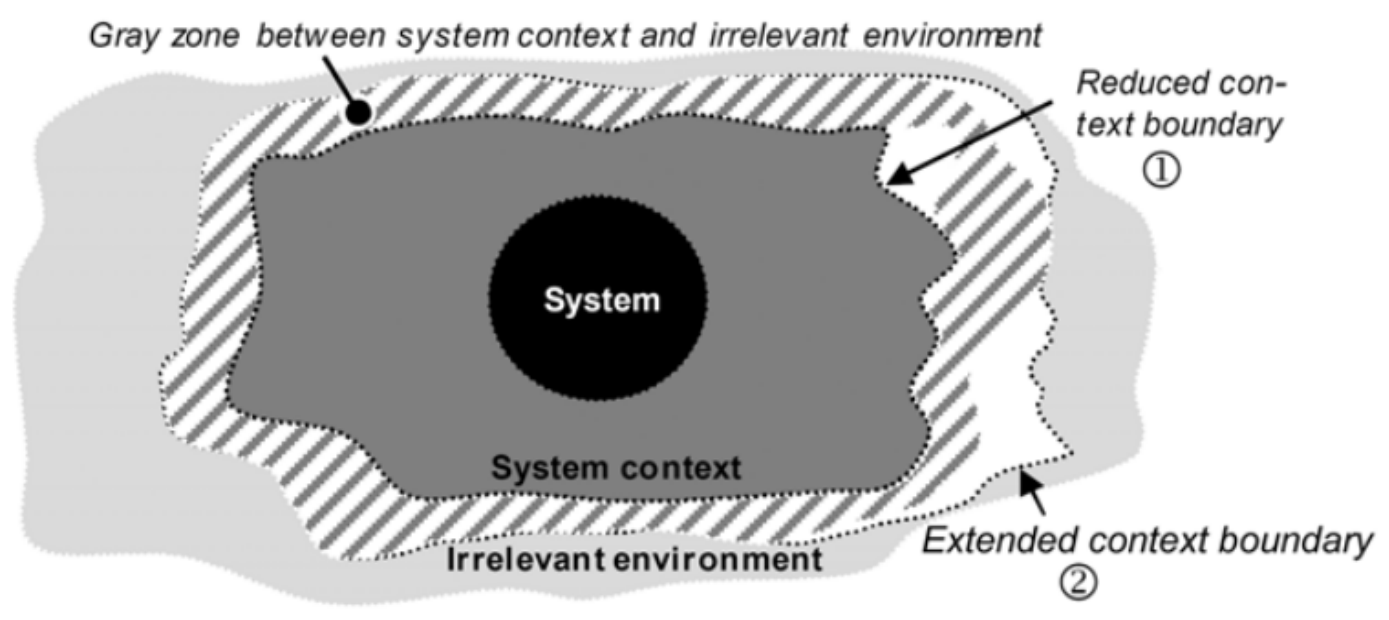


Figure: Gray zone between system ctx and irrelevant env[2]

[2] (Pohl 2010)

Requirement sources

Three main sources of requirements

- **Stakeholders** – people or organizations that directly or indirectly influence the requirements of a system.
  - Examples: users, operators, customers, architects, developers, testers.
- **Documents** – existing materials that provide relevant requirements or constraints.
  - Examples: standards, legal texts, internal policies, legacy error reports.
- **Existing systems** – legacy, predecessor, or competing systems that can be analyzed to identify desired extensions or changes.

The Kano model I

Kano model [Kano et al., 1984]

Understanding how requirements influence **stakeholder satisfaction** helps to prioritize and elicit them effectively.

- **Dissatisfiers** – self-evident or taken-for-granted properties (subconscious knowledge)
  - Must be fulfilled — otherwise cause dissatisfaction.
  - Full satisfaction does not increase happiness, only prevents discontent.
- **Satisfiers** – explicitly demanded properties (conscious knowledge)
  - Stakeholders are aware of them and request them explicitly.
  - Missing satisfiers directly reduce acceptance and satisfaction.
- **Delighters** – unexpected, innovative properties (unconscious knowledge)
  - Surprising features discovered only during use.
  - Create delight and excitement, but over time become satisfiers or dissatisfiers.

The Kano model II

- Delighters often become satisfiers or dissatisfiers over time; consider all three categories when eliciting requirements.

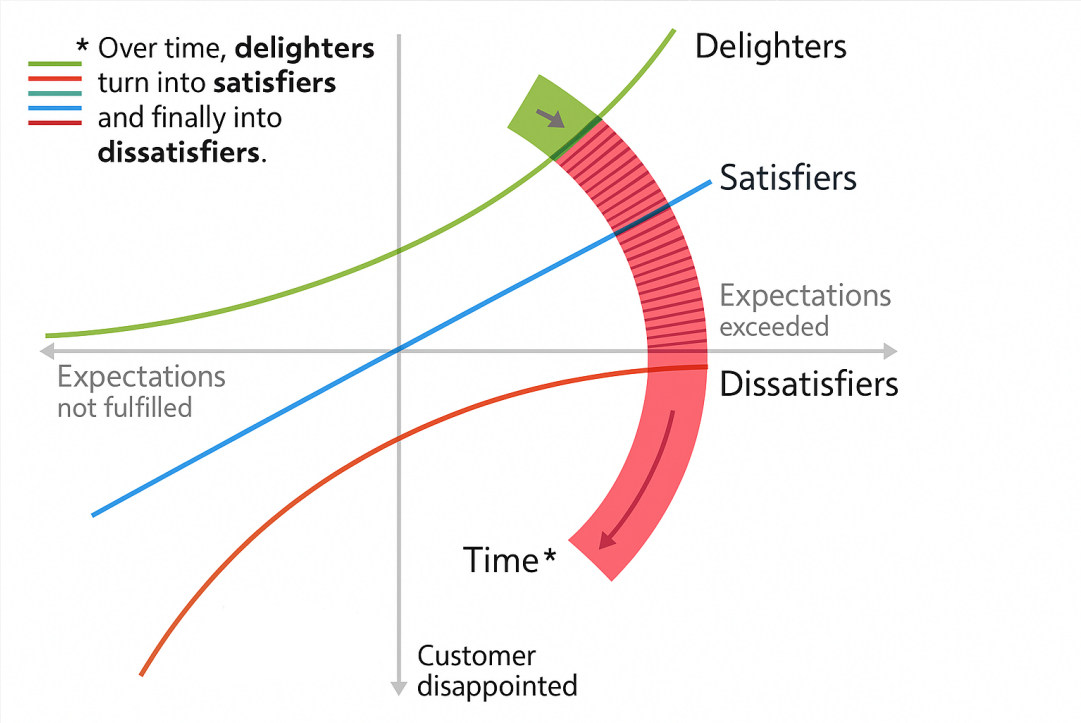


Figure: Kano model diagram[3]

[3] (Pohl 2010)

Requirement elicitation techniques

- Purpose: to **identify** and **understand** the requirements of a system from various sources (unconscious and conscious).
- No universal method.
- Risk factors: stakeholders' communication skills, domain knowledge, organizational culture, etc.
- Common techniques:
  - Survery techniques: Interviews, Questionnaires
  - Creativity techniques like: brainstorming/anti-branstorming sessions
  - Document-centric techniques:
    - System archaeology
    - Perspective-based reading
    - Requirements reuse
  - Observation techniques: field observation, apprenticing
  - Other useful techniques: mind mapping, prototyping, workshops

Wrocław University of Science and Technology

Requirements specification

Requirements specification

Requirements Specification

A requirements specification is a systematically represented collection of requirements, typically for a system or component, that satisfies given criteria.

- Basis of system design and development
- Binding between customer and developer
- Complexity requires structured approach

Criteria of a good **requirements specification** (ISO/IEC/IEEE 29148:2011):

- Unambiguity & consistency
- Completeness
- Modifiability & extendibility
- Traceability
- Clear structure

Criteria of a good requirement: agreed, unambiguous, necessary, consistend, verifiable, feasible, traceable, complete, understandable.

Requirement specification techniques

User and system requirements

- **User requirements** – written in natural language, often supported by diagrams and tables.
- **System requirements** – may include structured text, forms, graphical or mathematical notations.
- Should describe **functional and non-functional** aspects clearly for non-technical stakeholders.
- Should focus on **external behavior**, avoiding implementation or architecture details.

Notation	Description
Natural language	Simple sentences, each representing one requirement.
Structured natural language	Standardized templates or forms capturing requirement aspects.
Graphical notations	UML diagrams (use case, sequence) with textual annotations.
Mathematical specifications	Formal, unambiguous notations (e.g. set theory), rarely used by customers.

Glossary design

Why a glossary is essential

- Different interpretations of terms often cause **conflicts** in requirements engineering.
- A **shared vocabulary** ensures that all stakeholders understand terms consistently.
- A glossary defines:
  - Technical and domain-specific terms
  - Abbreviations and acronyms
  - Everyday terms with specific meanings
  - Synonyms (different terms, same meaning)

Rules for glossary management

- **Single source of truth** — one central, valid, and accessible glossary.
- **Continuous maintenance** throughout the project.
- **Mandatory usage** by all participants.
- **Include sources** of term definitions.
- **Stakeholder approval** ensures contextual correctness.
- **Consistent structure** — use a standard template for each entry.

Natural language implications

- Natural language (NL) is the most common medium for requirements specification. Everyone (almost) understands it.

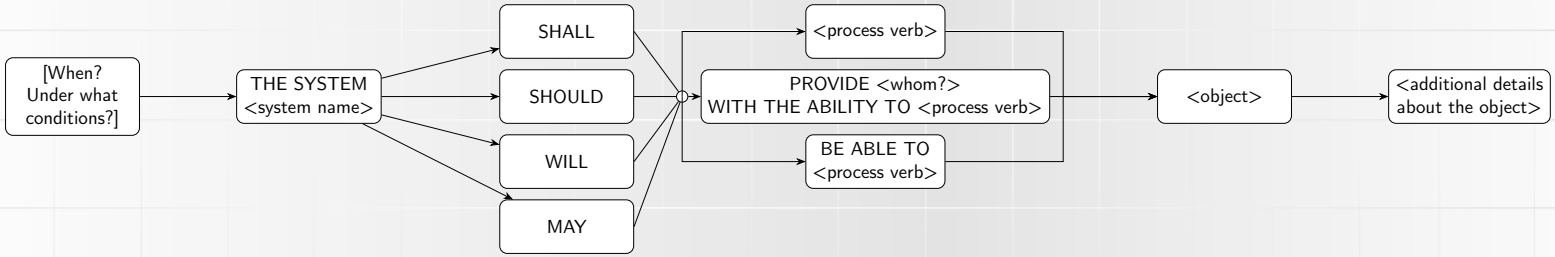
Issues of NL

- **Nominalization** – sometimes nouns define long and complex processes
  - Examples: "system update", "system shutdown", "data processing", etc.
- **Nouns with missing reference** – vague terms
  - Example: "The message should be displayed in the component". What component? What message?
- **Universal quantifiers** – words like "all", "any", "every", "always", etc. can lead to overgeneralization
  - Example: "The system shall always validate all user input."
- **Incomplete statements** – missing conditions or exceptions
  - Example: "The restaurant system shall offer all beverages to a registered guest over the age of 18 years."

Requirements templates

- **Step 1** Legal obligation – use modal verbs like: *shall*, *should*, *will*, *may*
- **Step 2** The Requirement Core – functionality itself
- **Step 3** Activity of the system: *autonomous activity*, *user interaction*, *interface call*
- **Step 4** Objects – additional details
- **Step 5** Logical and termporal conditions – conditions

The complete requirements template with conditions:



Examples of Requirement Sentences

System	Modal verb	Proc. verb	Object	Additional details
The system	shall	store	user login data	securely in the database
The system	shall	send	a confirmation email	after successful registration
The application	should	display	error messages	in the user's selected language
The server	will	generate	performance reports	every 24 hours
The web portal	may	allow	administrators	to export data as CSV files
The device	shall	restart	automatically	in case of a system failure
The mobile app	should	provide	users	with the ability to reset their password

Each sentence follows the canonical structure: *[When / Condition] + The system + [shall/should/may] + [process verb] + [object] + [details]*.

Model-based requirements specification

Use case diagrams

Use Cases

Use cases were first proposed in [Jacobson et al. 1992] as a model-based technique to **document the functionalities** of a planned or existing system.
The use case approach is based on two concepts that are used in conjunction with one another:

- Use Case Diagram
- Use case specifications

Use Case Diagram

Use Case Diagrams (UCDs) describe the **functional behavior of a system** from the user's perspective. They show how **actors** interact with **use cases** within system boundaries.

Use-case diagram elements

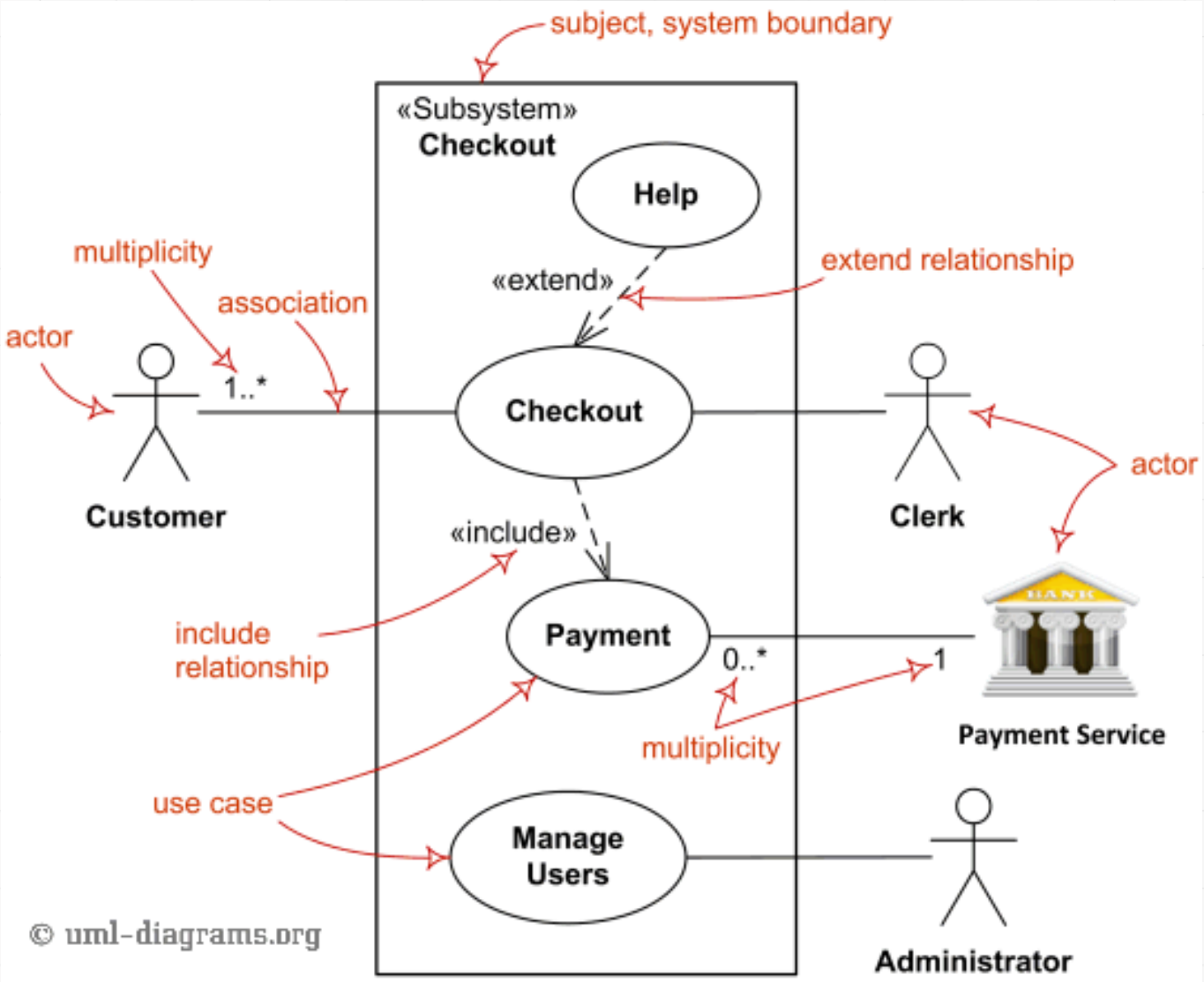


Figure: Use case diagram elements[4]

[4] https://uml-diagrams.com

Use Case Modeling (UML)

Key categories

1. **Use cases** – shown as *ovals*, each naming a system function. The name may also be placed below the oval.
2. **Actors** – external entities (people or systems) interacting with the system. Depicted as stick figures (or icons, or rectangles stereotyped with «actor»).
3. **System boundary** – rectangle enclosing all use cases; separates internal and external parts.

Relations between elements

1. **Include** – one use case *includes* another to reuse its behavior. Example: "Navigate to destination" *includes* "Input destination".
2. **Extend** – one use case *extends* another conditionally at a defined *extension point*. Example: "Download traffic info" *extends* "Navigate to destination" if "Avoid congestion".
3. **Generalization** – defines inheritance between actors or use cases. Example: "Service mechanic" and "Customer support" → "Employee".
4. **Communication links** – associations between actors and use cases where interaction occurs.

Use-case diagram example

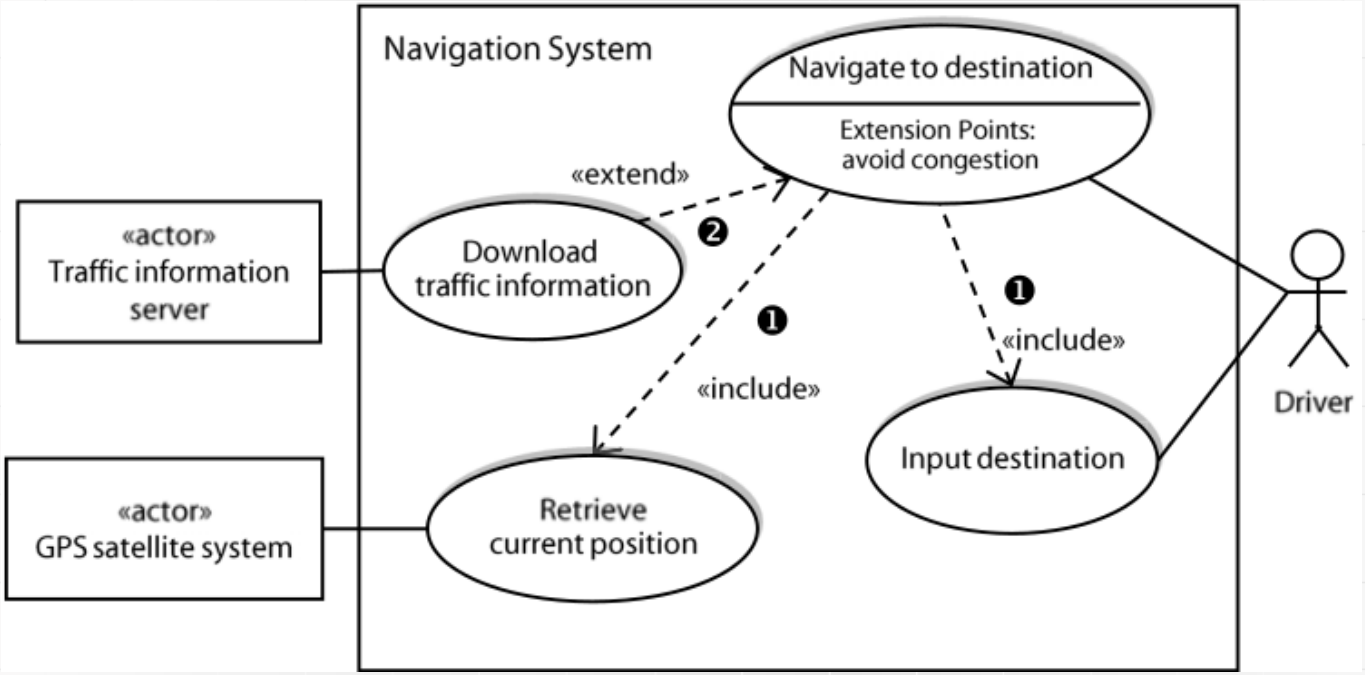


Figure: Use case diagram example[5]

[5] (Pohl 2010)

Use case specifications

Use Case Diagrams show only **relations and scope**, not internal behavior. To document the detailed interaction between an actor and the system, each use case must be described

- **textually using a specification template**
- optionally **complemented by behavioral models**: activity (workflow), sequence (message exchange), state machine (lifecycle).

.

Use-case specification example

Section	Content
Designation	UC-12-37
Name	Navigate to destination
Authors	John Smith, Sandra Miller
Priority	Importance for system success: **high** Technological risk: **high**
Criticality	High
Source	C. Warner (domain expert for navigation systems)
Responsible	J. Smith
Description	The driver of the vehicle types the name of the destination. The navigation system guides the driver to the desired destination.
Trigger event	The driver wishes to navigate to his destination.
Actors	Driver, traffic information server, GPS satellite system
Pre-condition	The navigation system is activated.
Post-condition	The driver has reached his destination.
Result	Route guidance to the destination
Main scenario	1. The navigation system asks for the desired destination. 2. The driver enters the desired destination. 3. The navigation system pinpoints the destination in its maps. 4. On the basis of the current position and the desired destination, the navigation system calculates a suitable route. 5. The navigation system compiles a list of waypoints. 6. The navigation system shows a map of the current position and shows the route to the next waypoint. 7. When the last waypoint is reached, the navigation system shows "destination reached" on the screen.
Alternative scenario	4a. Calculation of the route must honor traffic information and avoid traffic congestions. 4a1. The navigation system queries the server for updated traffic information. 4a2. The navigation system calculates a route that does not contain any traffic congestions.
Exception scenarios	Trigger event: The navigation system does not receive a GPS signal from the GPS satellite system.
Qualities	→ QR.04 (reaction time upon user input) → QR.15 (operating comfort) *(QR = quality requirements)*

Wrocław University of Science and Technology

Requirements validation

Wrocław University of Science and Technology

Validation principles

Requirements validation

Requirements validation is the process of verifying whether the defined requirements:

- accurately reflect the stakeholders' needs and expectations,
- are complete, consistent, unambiguous, and feasible,
- and can be approved as a valid basis for subsequent development activities such as design, implementation, and testing.

- Principle 1: Involvement of the correct stakeholders
- Principle 2: Separating the Identification and the Correction of errors
- Principle 3: Validation from different views
- Principle 4: Adequate change of documentation type
- Principle 5: Construction of development artifacts
- Principle 6: Repeated validation

Validation techniques

Goal

Requirements validation ensures that the documented requirements are **correct, complete, and agreed upon** before design and implementation begin. It helps identify **ambiguities, omissions, and contradictions** early.

Manual review techniques

- **Commenting** – peer review of individual requirements to detect unclear or incorrect statements.
- **Inspection** – structured group review with defined phases (planning, overview, error detection, consolidation). Roles: organizer, moderator, author, reader, inspectors, minute-taker.
- **Walk-through** – lightweight group discussion led by the author, used to build shared understanding and identify issues.

Complementary techniques

- **Perspective-based reading** – each reviewer inspects requirements from a different viewpoint (user, architect, tester, documentation, agreement).
- **Validation through prototypes** – stakeholders interact with a prototype to compare expectations with implementation. Types: throw-away vs. evolutionary.
- **Checklists** – structured lists of validation questions ensuring consistency, completeness, and comparability of results.

Requirements management

Requirements prioritization techniques I

Purpose

Prioritization determines the **relative importance and implementation order** of requirements. It helps allocate resources effectively and supports release planning.

Typical prioritization criteria

- **Business value / Importance**
- **Cost of implementation**
- **Risk** (technical, market, schedule)
- **Damage of failure**
- **Volatility** (likelihood of change)
- **Implementation effort / duration**

Common prioritization techniques

- **Ranking** – stakeholders sort requirements by importance.
- **Top-N** – select and rank the most important (e.g. top ten) requirements.
- **Single-criterion classification** – *Mandatory / Optional / Nice-to-have* (IEEE 830).
- **MoSCoW method** –
  - M — Must have
  - S — Should have
  - C — Could have
  - W — Won't have (this time)
- **Kano classification** – Dissatisfiers, Satisfiers, Delighters.
- **Wiegers matrix** – analytical approach combining *benefit, cost, risk, detriment*.

Traceability and change management

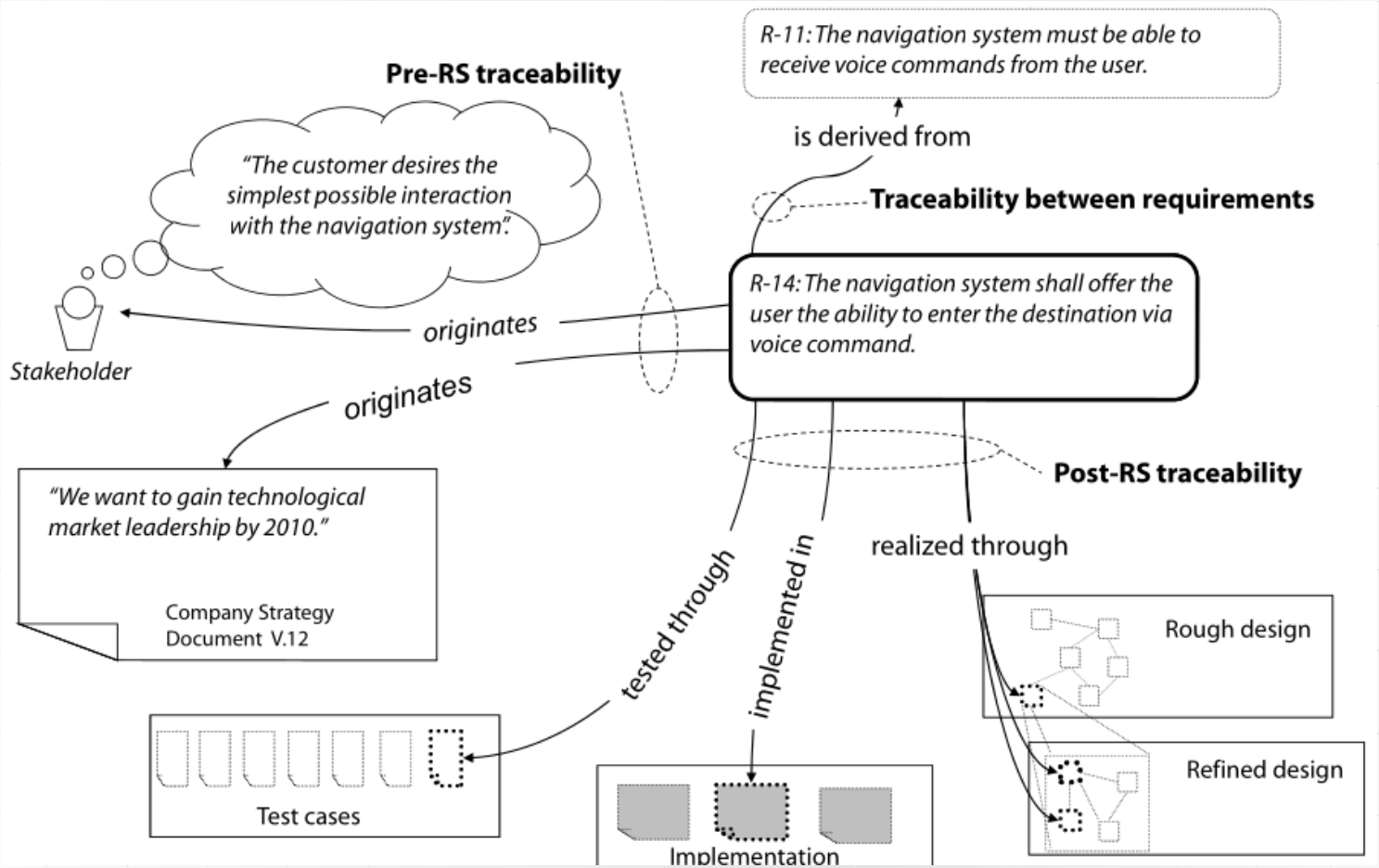
Traceability

The traceability of a requirement is the ability to trace the requirements over the course of the entire life cycle of the system

Types of Traceability

- pre-requirements-specification (pre-RS)
- post-reqirements-specification (post-RS)
- traceability between requirements

Traceability types: example



Wrocław University of Science and Technology

It's all for today!